Deakin University

Engineering AI Solutions

OnTrack Submission

Task 4.1P

Submitted By:
soumith samuel Kalapala
s225592148
2026/04/19 20:07

Tutor:
James Martin

April 19, 2026

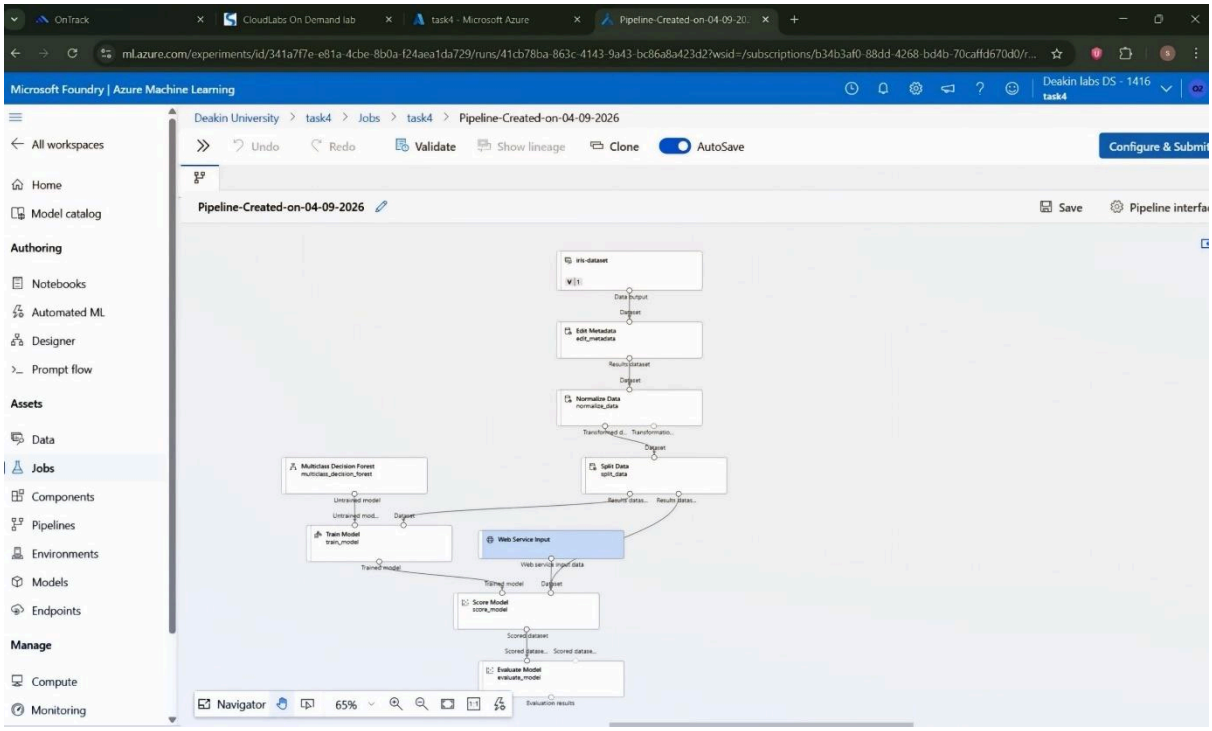
SIT788 – Engineering AI Solutions

Task 4.1P: MLOps and Model Deployment in Azure

Student name: SOUMITH SAMUEL KALAPALA
Student ID: 225592148

1. Pipeline Implementation in Azure ML Designer

The machine learning pipeline was created using Azure ML Designer, where a sequence of modules was configured to preprocess the data, train the model, and evaluate its performance. The pipeline began with the Iris dataset, which was passed through an Edit Metadata module to explicitly convert the Species column into a categorical variable. Prior to this, the Id column was excluded to prevent data leakage, as required by the assignment.



Following metadata adjustment, the dataset was normalized using Z-score normalization to scale all numerical features, including SepalLengthCm, SepalWidthCm, PetalLengthCm, and PetalWidthCm. The processed dataset was then split into training and testing subsets using a 70/30 ratio with stratified sampling based on the Species column and a fixed random seed of 42 to ensure reproducibility.

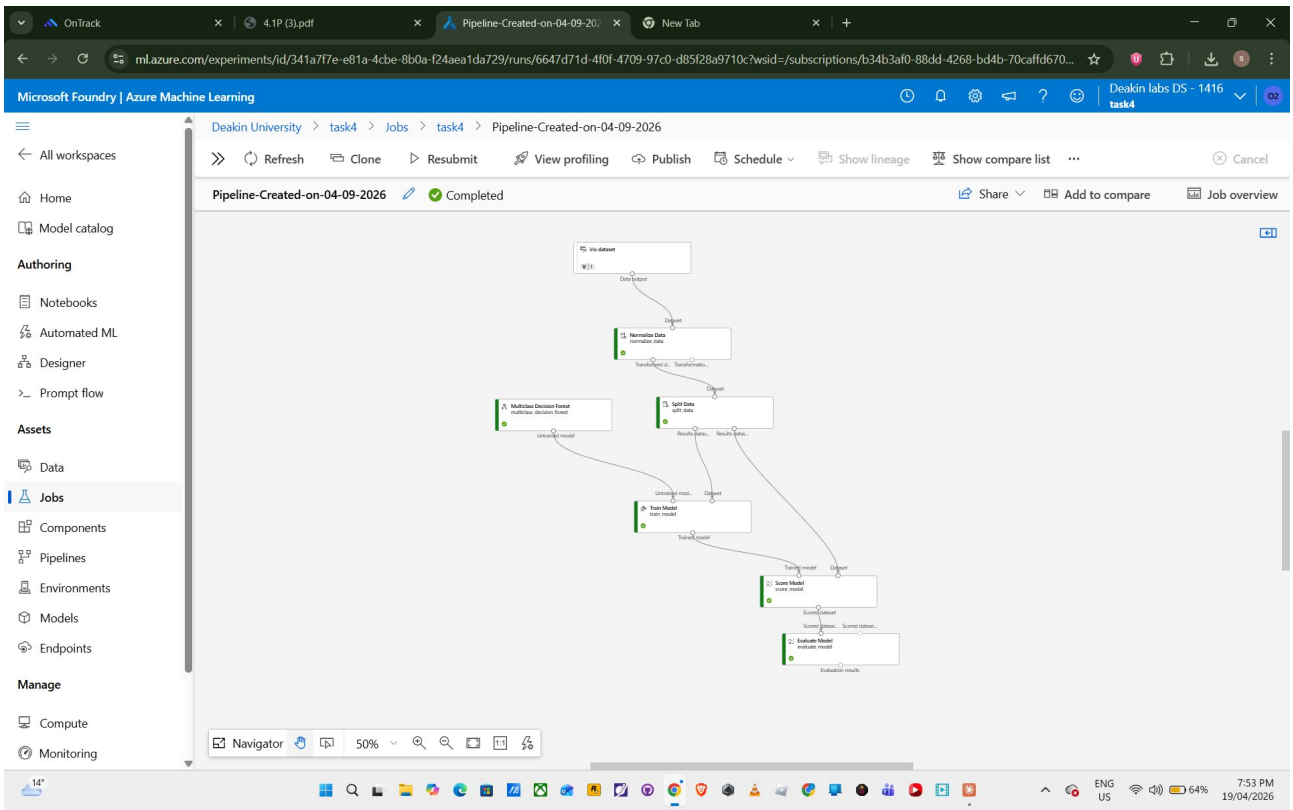
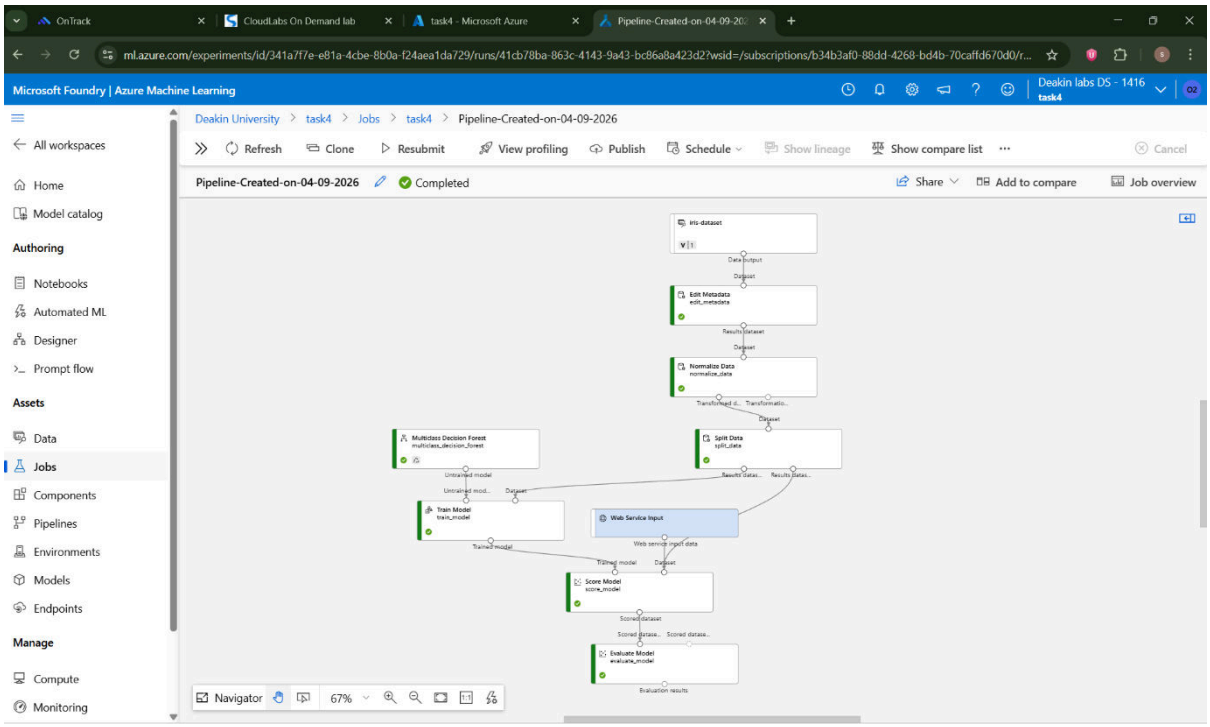


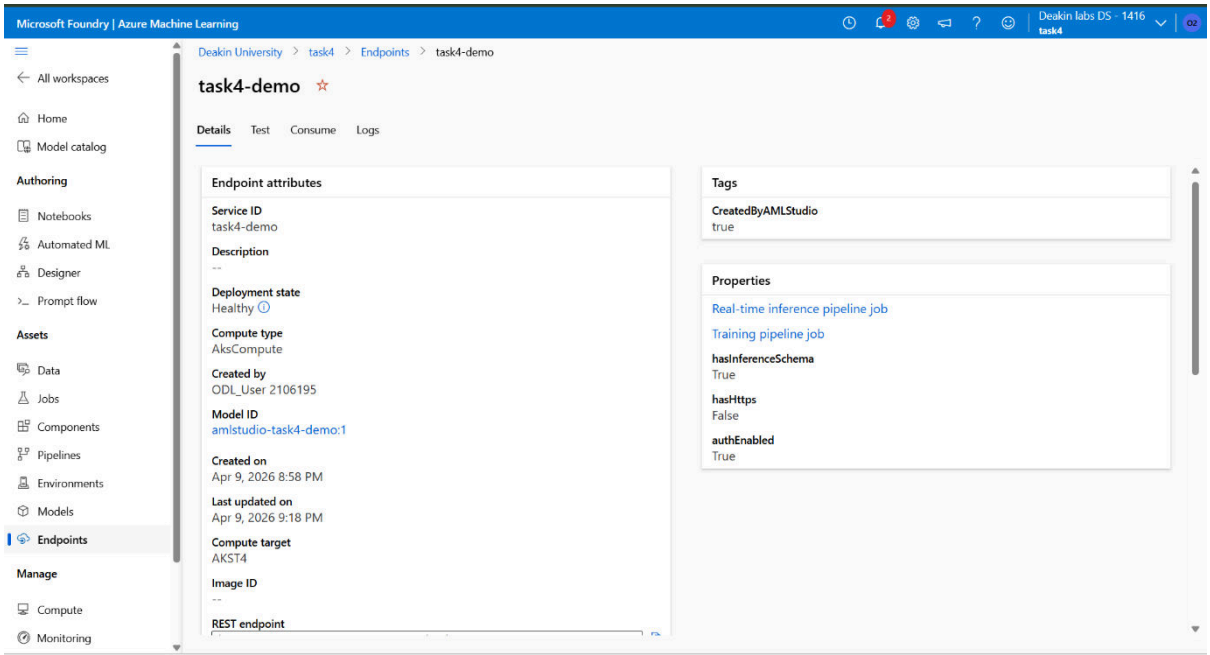
Figure 1: Training Pipeline in Azure ML Designer showing the complete workflow from Iris Dataset through Normalize Data, Split Data, Multiclass Decision Forest, Train Model, Score Model to Evaluate Model.



A Multiclass Decision Forest algorithm was selected as the classification model, and the training process was configured with Species as the label column. The model was evaluated using the Evaluate Model module, confirming its effectiveness in classifying iris species.

2. Inference Pipeline and Deployment

After successfully training and evaluating the model, a real-time inference pipeline was created to enable deployment as a web service. The inference pipeline includes a Web Service Input module for accepting external data and a Score Model module to generate predictions.



The model was deployed to an Azure Kubernetes Service (AKS) cluster, providing a scalable and production-ready environment. The successful deployment is confirmed by the endpoint status being marked as “Healthy,” and the availability of REST API details for external access.

3. API Testing and Validation

To validate the deployed model, a Python client script was used to send a request to the REST API endpoint. The script included the endpoint URL and API key along with a JSON payload structured according to the required schema. Dummy values for Id and Species were included to avoid schema mismatch errors

Python API Request Code Snippet

```
import requests
import json

# API Configuration
API_KEY = "x9qfxgCLbh6wi76YTQA9pQfZNW5lRsOw"
ENDPOINT_URL = "http://20.248.234.225:80/api/v1/service/task4-demo/score"

# Request Headers
headers = {
    "Content-Type": "application/json",
    "Authorization": f"Bearer {API_KEY}"
}

# Sample Iris Data for Prediction
data = {
    "Inputs": {
        "input1": [
            {
                "SepalLengthCm": 5.1,
                "SepalWidthCm": 3.5,
                "PetalLengthCm": 1.4,
                "PetalWidthCm": 0.2,
                "Species": "Iris-setosa"
            }
        ]
    },
    "GlobalParameters": {}
}

# Send POST Request
response = requests.post(
    ENDPOINT_URL,
    headers=headers,
    json=data
)

# Print Response
print("Status Code:", response.status_code)
print("Prediction Response:")
print(json.dumps(response.json(), indent=2))
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\soumi> cd downloads
PS C:\Users\soumi\downloads> python task4_client.py
Status Code: 200
Prediction Response:
{
  "Results": {
    "WebServiceOutput0": [
      {
        "SepalLengthCm": 5.1,
        "SepalWidthCm": 3.5,
        "PetalLengthCm": 1.4,
        "PetalWidthCm": 0.2,
        "Species": "Iris-setosa",
        "Scored Probabilities_Iris-setosa": 0.0,
        "Scored Probabilities_Iris-versicolor": 0.375,
        "Scored Probabilities_Iris-virginica": 0.625,
        "Scored Labels": "Iris-virginica"
      }
    ]
  }
}
PS C:\Users\soumi\downloads>
```

The script executed successfully, returning a status code of 200 and a JSON response containing predicted probabilities and the final classification label. The model predicted Iris-virginica with the highest probability, demonstrating correct functionality of the deployed service.

Feature Name	Type	Data Type	Range / Possible Values	Description
Id	Numerical	Integer	1 – 150	Unique identifier for each sample (excluded before training to avoid data leakage)
SepalLengthCm	Numerical	Float	4.3 – 7.9 cm	Length of the sepal in centimeters
SepalWidthCm	Numerical	Float	2.0 – 4.4 cm	Width of the sepal in centimeters
PetalLengthCm	Numerical	Float	1.0 – 6.9 cm	Length of the petal in centimeters
PetalWidthCm	Numerical	Float	0.1 – 2.5 cm	Width of the petal in centimeters
Species	Target	Categorical	Iris-setosa, Iris-versicolor, Iris-virginica	Target variable representing the flower species

4. Feature Importance

The Multiclass Decision Forest model determines feature importance based on how frequently each feature is used to split the data across decision trees. In this implementation, petal-related features, particularly PetalLengthCm and PetalWidthCm, contributed most significantly to the model's predictions, while sepal features had comparatively lower importance. This aligns with known characteristics of the Iris dataset.

5. Conclusion

In conclusion, this task successfully demonstrates the complete lifecycle of a machine learning solution, from data preprocessing and model training to deployment and real-time inference. The use of Azure Machine Learning Studio enabled the development of a robust and scalable pipeline, while deployment to AKS ensured production-grade performance. The successful API testing using a Python client script confirms that the model is operational and capable of delivering predictions in real time. This task effectively highlights the practical implementation of MLOps principles in a cloud-based environment.

```python
import requests
import json

# API Configuration
API_KEY = "x9qfxgCLbh6wi76YTQA9pQfZNW5lRsOw"
ENDPOINT_URL = "http://20.248.234.225:80/api/v1/service/task4-demo/score"

# Request headers
headers = {
    "Content-Type": "application/json",
    "Authorization": f"Bearer {API_KEY}"
}

# Sample Iris data for prediction
data = {
    "Inputs": {
        "input1": [
            {
                "SepalLengthCm": 5.1,
                "SepalWidthCm": 3.5,
                "PetalLengthCm": 1.4,
                "PetalWidthCm": 0.2,
                "Species": "Iris-setosa"
            }
        ]
    },
    "GlobalParameters": {}
}

# Send request
response = requests.post(ENDPOINT_URL, headers=headers, json=data)

# Print result
print("Status Code:", response.status_code)
print("Prediction Response:")
print(json.dumps(response.json(), indent=2))
```

```python
import requests
import json

# API Configuration
API_KEY = "x9qfxgCLbh6wi76YTQA9pQfZNW5lRsOw"
ENDPOINT_URL = "http://20.248.234.225:80/api/v1/service/task4-demo/score"

# Request headers
headers = {
    "Content-Type": "application/json",
    "Authorization": f"Bearer {API_KEY}"
}

# Sample Iris data for prediction
data = {
    "Inputs": {
        "input1": [
            {
                "SepalLengthCm": 5.1,
                "SepalWidthCm": 3.5,
                "PetalLengthCm": 1.4,
                "PetalWidthCm": 0.2,
                "Species": "Iris-setosa"
            }
        ]
    },
    "GlobalParameters": {}
}

# Send request
response = requests.post(ENDPOINT_URL, headers=headers, json=data)

# Print result
print("Status Code:", response.status_code)
print("Prediction Response:")
print(json.dumps(response.json(), indent=2))
```